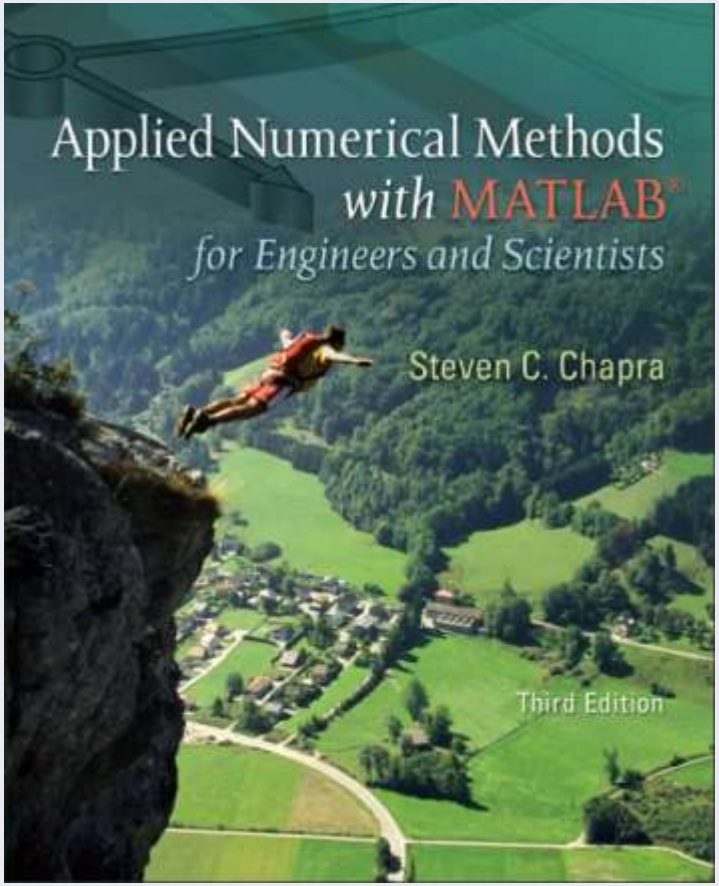


Part 3
Chapter 10

LU Factorization

PowerPoints organized by Dr. Michael R. Gustafson II, Duke University

All images copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Chapter Objectives

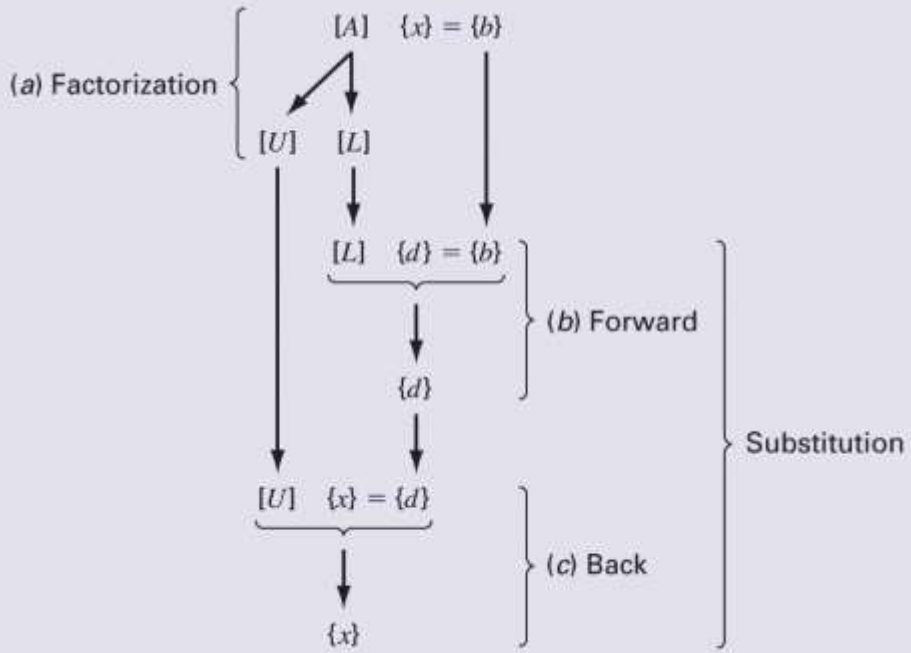
- Understanding that *LU* factorization involves decomposing the coefficient matrix into two triangular matrices that can then be used to efficiently evaluate different right-hand-side vectors.
- Knowing how to express Gauss elimination as an *LU* factorization.
- Given an *LU* factorization, knowing how to evaluate multiple right-hand-side vectors.
- Recognizing that Cholesky's method provides an efficient way to decompose a symmetric matrix and that the resulting triangular matrix and its transpose can be used to evaluate right-hand-side vectors efficiently.
- Understanding in general terms what happens when MATLAB's backslash operator is used to solve linear systems.

LU Factorization

- Recall that the forward-elimination step of Gauss elimination comprises the bulk of the computational effort.
- *LU* factorization methods separate the time-consuming elimination of the matrix [*A*] from the manipulations of the right-hand-side [*b*].
- Once [*A*] has been factored (or decomposed), multiple right-hand-side vectors can be evaluated in an efficient manner.

LU Factorization

- *LU* factorization involves two steps:
  - Factorization to decompose the [*A*] matrix into a product of a lower triangular matrix [*L*] and an upper triangular matrix [*U*]. [*L*] has 1 for each entry on the diagonal.
  - Substitution to solve for {*x*}
- Gauss elimination can be implemented using *LU* factorization



Gauss Elimination as *LU* Factorization

- [*A*]{*x*}={*b*} can be rewritten as [*L*][*U*]{*x*}={*b*} using *LU* factorization.
- The *LU* factorization algorithm requires the same total flops as for Gauss elimination.
- The main advantage is once [*A*] is decomposed, the same [*L*] and [*U*] can be used for multiple {*b*} vectors.
- MATLAB's `lu` function can be used to generate the [*L*] and [*U*] matrices:

```
[L, U] = lu(A)
```

Gauss Elimination as *LU* Factorization (cont)

- To solve [*A*]{*x*}={*b*}, first decompose [*A*] to get [*L*][*U*]{*x*}={*b*}
- Set up and solve [*L*]{*d*}={*b*}, where {*d*} can be found using *forward* substitution.
- Set up and solve [*U*]{*x*}={*d*}, where {*x*} can be found using *backward* substitution.
- In MATLAB:

```
[L, U] = lu(A)
d = L\b
x = U\d
```

Cholesky Factorization

- Symmetric systems occur commonly in both mathematical and engineering/science problem contexts, and there are special solution techniques available for such systems.
- The *Cholesky factorization* is one of the most popular of these techniques, and is based on the fact that a symmetric matrix can be decomposed as $[A]= [U]^T[U]$, where T stands for transpose.
- The rest of the process is similar to *LU* decomposition and Gauss elimination, except only one matrix, $[U]$, needs to be stored.

MATLAB

- MATLAB can perform a Cholesky factorization with the built-in `chol` command:

```
U = chol(A)
```

- MATLAB's left division operator \ examines the system to see which method will most efficiently solve the problem. This includes trying banded solvers, back and forward substitutions, Cholesky factorization for symmetric systems. If these do not work and the system is square, Gauss elimination with partial pivoting is used.